

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1718

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 10%

Duration : 45

Minutes QUESTIONS: 2 Total Marks:20

Annexure A

Descriptive Written Test

Code of Conduct:

I ____A Bharath / 192512384 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A . Bharath

Question 1: Smart Home Automation System Aim

To study different types of intelligent agents and analyze their role in a smart home automation system for improving comfort, energy efficiency, and security.

Problem Statement

A smart home automation system manages **lighting, temperature, and security** based on user behavior and environmental conditions. The system receives real-time inputs such as **motion detection, temperature changes, and time of day**, while learning user preferences over time. The objective is to understand how different intelligent agents work in this environment and determine the most suitable approach.

Description 1. Simple Reflex Agent

- Makes decisions based only on the current input.
- Example:
 - Motion detected → Turn on lights.
 - No motion → Turn off lights.

Advantage: Fast and simple.

Disadvantage: Cannot remember previous events.

2. Model-Based Agent

- Maintains an internal model of the environment.
- Uses current and past information.

Example:

- Knows whether someone is already inside the room before switching lights.

Advantage: Better decisions.

Disadvantage: Requires memory.

3. Goal-Based Agent

- Selects actions that achieve a specific goal.

Example:

- Goal: Maintain room temperature at **24°C**.
- Turns AC or heater ON/OFF until the goal is achieved.

Advantage: Goal-oriented.

Disadvantage: More computation.

4. Utility-Based Agent

- Chooses the action that gives the highest overall benefit.

Example:

- Balances comfort, electricity usage, and security simultaneously.

Advantage: Best overall performance.

Disadvantage: Complex implementation.

Best Approach

A **Hybrid Agent (Model-Based + Goal-Based + Utility-Based)** is the most effective because it:

- Learns user preferences.
 - Maintains room history.
 - Achieves desired comfort.
 - Minimizes electricity consumption.
 - Maximizes home security.
-

Python Code

```
temperature = 30
```

```
motion = True
```

```
time = "Night"
```

```
if motion:
```

```
    light = "ON" else:
```

```
    light = "OFF"
```

```
if temperature > 26:
```

```
    ac = "ON" else:
```

```
    ac = "OFF"
```

```
if time == "Night":
```

```
    security = "ARMED" else:
```

```
    security = "DISARMED"
```

```
print("Light:", light) print("AC:",
```

```
ac) print("Security:", security)
```

Output

Light: ON

AC: ON

Security: ARMED

```
import heapq

graph = {
    'S': [('A', 2), ('B', 1)],
    'A': [('C', 2)],
    'B': [('D', 3)],
    'C': [('G', 3)],
    'D': [('G', 2)],
    'G': []
}

def ucs(start, goal):
    pq = [(0, start, [start])]
    visited = set()

    while pq:
        cost, node, path = heapq.heappop(pq)

        if node == goal:
            print("Path:", " -> ".join(path))
            print("Cost:", cost)
            return

        if node in visited:
            continue

        visited.add(node)

        for neighbor, weight in graph[node]:
            if neighbor not in visited:
```

Path: S -> B -> D -> G
Cost: 6

=== Code Execution Successful ===

Conclusion

A hybrid intelligent agent provides better comfort, energy efficiency, and security by combining memory, goal achievement, and utility optimization.

Question 2: Robot in an Unknown Maze Aim

To study **Uniform Cost Search (UCS)** and **Iterative Deepening Search (IDS)** for solving an unknown maze and identify the most suitable intelligent agent.

Problem Statement

A robot is placed in an unknown maze and must find the exit without prior knowledge of the maze. The robot should use **Uninformed Search Algorithms** such as **Uniform Cost Search (UCS)** and **Iterative Deepening Search (IDS)** to explore the maze and reach the goal efficiently.

Description Uniform Cost Search (UCS)

- Expands the node with the lowest path cost.
- Guarantees the minimum-cost path.

Advantages

- Optimal solution.
- Suitable for different movement costs.

Disadvantages

- High memory usage.
 - Slower for large search spaces.
-

Iterative Deepening Search (IDS)

- Performs repeated depth-limited searches.
- Increases the depth limit until the goal is found.

Advantages

- Low memory usage.
- Complete search algorithm.

Disadvantages

- Repeats node expansions.
 - Slower than BFS for shallow goals.
-

Time and Space Complexity Algorithm Time Complexity Space Complexity

UCS $O(b^{(1+C^*/\epsilon)})$ $O(b^{(1+C^*/\epsilon)})$

IDS $O(b^d)$ $O(bd)$

Where:

- **b** = branching factor
- **d** = depth of goal
- **C*** = optimal path cost
- **ε** = minimum step cost

Intelligent Agent

A **Goal-Based Agent** is most suitable because:

- The robot's goal is to reach the exit.
- It plans actions to achieve that goal.
- It adapts as it explores the unknown maze.

Best Combination

Goal-Based Agent + Uniform Cost Search (UCS) Reason:

- Finds the least-cost path.
- Handles varying movement costs.
- Guarantees an optimal solution.

Python Code (Uniform Cost Search) import

heapq

graph = {

 'S': [('A', 2), ('B', 1)],

```
'A': [('C', 2)],  
'B': [('D', 3)],  
'C': [('G', 3)],  
'D': [('G', 2)],  
'G': []  
}
```

```
def ucs(start, goal):
```

```
    pq = [(0, start, [start])]
```

```
    visited = set()
```

```
    while pq:
```

```
        cost, node, path = heapq.heappop(pq)
```

```
        if node == goal:
```

```
            print("Path:", " -> ".join(path))
```

```
    print("Cost:", cost)    return
```

```
        if node in visited:
```

```
            continue
```

```
        visited.add(node)
```

```
        for neighbor, weight in graph[node]:
```

```
            if neighbor not in visited:
```

```
heapq.heappush(pq, (cost + weight, neighbor, path + [neighbor]))
```

```
ucs('S', 'G')
```

Output

Path: S -> B -> D -> G

Cost: 6

<pre>temperature = 30 motion = True time = "Night" if motion: light = "ON" else: light = "OFF" if temperature > 26: ac = "ON" else: ac = "OFF" if time == "Night": security = "ARMED" else: security = "DISARMED" print("Light:", light) print("AC:", ac) print("Security:", security)</pre>	<pre>Light: ON AC: ON Security: ARMED === Code Execution Successful ===</pre>
---	--

Conclusion

For an unknown maze, a **Goal-Based Agent** combined with **Uniform Cost Search (UCS)** is the most effective solution because it guarantees the least-cost path while guiding the robot toward the exit. IDS is memory-efficient but may repeat searches, making UCS the better choice when optimality is important.